

Next.js 16 + Prisma
7
PostgreSQL &
Supabase

خطة تطوير نظام إدارة الصيدلية (Pharmacy Management System)

دليل النقل من مرحلة MVP إلى مستوى مهندس أول (Senior Enterprise Architecture)

الرؤية العامة: رفع جودة النظام لا يقتصر على كتابة ميزات إضافية، بل يركز على سلامة البيانات المالية والدوائية تحت الضغط العالي (Concurrency)، فصل طبقات الكود (Layered Architecture)، الأمان الصارم، والتوافق مع واقع الصيدليات والمستودعات الطبية.

1 معالجة التزامن وسباق البيانات (Concurrency & Race Conditions)

لماذا هذه النقطة محورية؟ إذا باع كاشيران آخر علبة دواء في نفس اللحظة، سيحدث بيع بالسالب (Overselling) بسبب قراءة الكمية قبل اكتمال الخصم.

- **قيود قاعدة البيانات (Database Check Constraints):** منع الكميات السالبة على مستوى PostgreSQL بشكل صارم حتى لو حدث خلل في كود التطبيق:

```
ALTER TABLE "Batch" ADD CONSTRAINT "batch_quantity_non_negative" CHECK ("quantity" ≥ 0)
```

- **القفل التنافسي (Pessimistic Locking / SELECT FOR UPDATE):** قفل أسطر الدفعات أثناء المعاملة البنكية والبيع داخل `$transaction` لضمان عدم تعديل المخزون من عملية أخرى حتى اكتمال المعاملة.

2 معمارية الكود وتطبيق النمط الطبقي (Layered Architecture)

- فصل المسؤوليات (Separation of Concerns): نقل منطق الـ FEFO والعمليات الحسابية من Server Actions إلى Services مستقلة، وفصل استعلامات قاعدة البيانات إلى Repositories.
- هيكلية المجلدات المقترحة:

```
src/
├── app/                # العرض والمفحات (UI & Routes)
├── lib/
│   ├── actions/       # بوابات التحقق وصلاحيات المستخدم (Entry Points)
│   ├── services/      # منطق الأعمال الصافي (SaleService, FEFOEngine)
│   ├── repositories/  # التعامل المباشر مع قاعدة البيانات (Data Access)
│   └── schemas/       # مخططات التحقق من المدخلات (Zod Schemas)
```

- التحقق الصارم عبر Zod و نمط النتيجة الموحد: اعتماد نمط `Result<T, E>` لمنع انهيار الخادم وجعل معالجة الأخطاء متوقعة تماماً في الواجهة.
- إزالة الـ `as any` : كتابة تعريفات Typescript دقيقة مع Prisma 7 لتجنب أخطاء وقت التشغيل (Runtime Errors).

3 منطق أعمال الصيدليات الواقعي (Domain Realities)

- سعر التكلفة مقابل سعر البيع (Cost Price vs Selling Price): إضافة حقل `costPrice` لكل دفعة قادمة من المورد، لحساب هامش الربح الصافي الفعلي لكل فاتورة بيع ($\text{Profit} = \text{SalePrice} - \text{BatchCostPrice}$).
- إدارة المرتجعات (Returns & Restocking): إتاحة استرجاع الأدوية وإعادةتها لنفس الدفعة الأصلية مع إنشاء سند إرجاع دقيق (Credit Note).
- الأدوية الخاضعة للرقابة (Controlled / Prescription Drugs): إضافة علم `requiresPrescription` للأدوية النفسية والمقيدة، وطلب اسم الطبيب ورقم الوصفة قبل تأكيد الصرف.
- نظام تنبيهات النواقص والانتهاء (Expiry & Low Stock Alerts): مهام دورية مجدولة (Cron Jobs) تفحص الأدوية التي تقترب صلاحيتها من 30/60/90 يوماً وتلك التي انخفض رصيدها عن الحد الأدنى.

4 سجل التدقيق والأمان المتقدم (Audit Logging & Security)

- سجل التتبع والمراجعة (Immutable Audit Log): تسجيل أي تعديل في الأسعار، أو تعديل يدوي في المخزون، أو إلغاء فواتير لمعرفة من قام بالعملية وتاريخها وقيمتها السابقة والجديدة.
- تحديد معدل الطلبات (Rate Limiting): حماية بوابات إرسال رسائل الـ OTP وتسجيل الدخول عبر Redis لمنع استنزاف رصيد الرسائل وهجمات Brute-Force.
- الصلاحيات الدقيقة (Granular RBAC): توزيع الصلاحيات (البيع، تعديل الأسعار، الاطلاع على الأرباح، إلغاء الفواتير) بدلاً من قصرها على خيارين فقط.

5 الاختبارات والمراقبة (Testing & Observability)

- اختبارات الوحدة لمحرك FEFO (Unit Testing): استخدام Vitest للاختبار توزيع السحب من الدفعات المتعددة، رفض الدفعات المنتهية، واختبار التراجع التلقائي (Rollback) عند فشل أي خطوة.
- التسجيل المهيكل (Structured Logging): استبدال console.log بنظام تسجيل مهيكل (Pino) لتسهيل فلترة العمليات في السيرفرات السحابية.

★ خطة التنفيذ المقترحة (Implementation Roadmap)

المرحلة	الأهداف الرئيسية	الأولوية والتأثير
المرحلة الأولى: هندسة الكود ومحرك FEFO	فصل كود المبيعات إلى SaleService ، كتابة Unit Tests ، خوارزمية FEFO، والتخلص من الـ as any .	عالية جداً (أساس المشروع)
المرحلة الثانية: سلامة قاعدة البيانات والتزامن	إضافة قيود CHECK على كميات الدفعات، تطبيق FOR UPDATE لمنع السباق، وإضافة حقل costPrice .	أمان مالي ومخزني
المرحلة الثالثة: سجل المراجعة والمرتجعات	بناء جدول AuditLog ، دعم فواتير الإرجاع، وإضافة تصنيفات الوصفات الطبية المعقدة.	متوسطة (مستوى متقدم)
المرحلة الرابعة: المراقبة ومعدل الطلبات	حماية الـ OTP بـ Rate Limiting ، وإعداد تسجيل الأحداث المهيكل (Structured Logging).	مستوى إنتاجي Enterprise

تم إعداد هذه الوثيقة وفق أفضل ممارسات هندسة البرمجيات لتطبيقات Next.js و PostgreSQL لعام 2026.